

Visualising Developer Interactions in Code Reviews

Daniel Bee

Royal Holloway, University of London
Egham, UK

Daniel.Bee.2022@live.rhul.ac.uk

DongGyun Han

Royal Holloway, University of London
Egham, UK

DongGyun.Han@rhul.ac.uk

Abstract

Code review is an auditing process in which all changes reviewed by someone other than the author of the changes. Because of its diverse benefits, as demonstrated by multiple studies, it has been widely adopted in both industry and open source projects. Although many studies have proposed techniques to recommend reviewers who can audit changes, there is limited support for comprehending the existing review pairs (the pairs of the change authors and reviewers). Without this understanding, development teams may struggle to distribute their workloads effectively. For example, a development team might overlook a concentrated workload on a specific developer. In this case, the overall development process can slow down. On the other hand, some developers submit code review requests (pull requests), but fail to receive attention from their colleagues.

To help developers understand the existing review pairs, we propose CRV (Code Review Visualiser). CRV visualises developers interactions during code reviews by quantifying their activities in a graph representation. In the graph, each node represents individual developer while the edges denote the relationships based on the discussions within the same code review requests. The size of node shows the number of total comments that a developer wrote in code review requests. The thickness of edge between two nodes represents the number of comments that two developers authored within the same code review requests. Since CRV provides time window control, developers can understand their code review activities across different time frames. We expect that CRV can help developers improve their code review practices by providing a better understanding of their current practices.

CCS Concepts

• **Software and its engineering** → **Maintaining software.**

Keywords

code review, visualisation, developer interactions

ACM Reference Format:

Daniel Bee and DongGyun Han. 2018. Visualising Developer Interactions in Code Reviews. In *Proceedings of IEEE/ACM International Conference on Software Engineering (ICSE '25)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE '25, April 27–May 3, 2025, Ottawa, Ontario, Canada

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Code review is a practice that all code changes are reviewed by one or more developers other than the change author themselves. Due to diverse benefits of code review, it has been widely adopted in both proprietary and open source projects. For example, previous studies have shown that it is useful for sharing knowledge and development context within a development team [1]. In addition, code review is helpful for improving software quality and design quality [7, 8]. Because of the diverse benefits, the practice has been widely adopted in both open and proprietary software projects. GitHub's pull request¹ is a representative example of code review.

One of the most challenging parts of code review is to find peers to review code changes. To mitigate the issue, many studies proposed reviewer recommendation techniques [10, 12]. These techniques recommend a group of developers for a given code review request. However, these techniques often limited to provide a guidance to individual code review requests rather than providing project wide guidances.

In this study, we propose a web service, CRV (Code Review Visualiser), that visualises developer interaction during pull request in GitHub. Especially, CRV visualises the frequency with which a developer involved in code reviews and how often a pair of developers discuss together within the same code review. Based on a given project, our technique provides a visualisation of developers within a given timeframe. We expect the visualisation helps a software development team to understand the performance of their code review practices over time. They potentially identify a pair of developers who are struggling work each other (e.g., a pair of developers who rarely communicate each other). We also expect the visualisation help to plan how to pair developers to work together. They probably group the developers who actively communicate each other for boosting development speed. Alternatively, they can group the developers who have rarely worked together for leveraging fresh synergy between new combination.

CRV consists of three components: crawler, migrator, and visualiser. Since the visualisation requires multiple pull requests to comprehend the interactions, requesting such a large amount of data from GitHub could potentially be burdensome and slow. To avoid generating excessive traffic and response users request quickly, the crawler component retrieves pull request data from GitHub using the GitHub GraphQL API². Once the crawling is done, the migrator component process the crawled data and store them into a database. Finally, the visualiser component visualises how developers interact to each other during code review.

We expect CRV can help developers improve their code review practices. For example, a development team can encourage a pair of developers to work together if the pair has collaborated on many

¹<https://docs.github.com/en/pull-requests>

²<https://docs.github.com/en/graphql>

code review requests. Alternatively, they can consider changing code review pairs to bring in a fresh perspective.

2 Developer Interactions

```

1 # Pull requests must be provided
2 prs = []
3 # The timeframe is optional parameters
4 timeframe = (start_date, end_date)
5
6 # The number of comments per developer.
7 nodes = {}
8 # The number of a developer pair within a PR.
9 links = {}
10
11 for pr in prs:
12     if timeframe is given and pr is not within the
13         timeframe:
14         continue
15     for comment in pr.comments:
16         if pr.author == comment.author:
17             continue
18
19     nodes[comment.author] += 1
20
21     author_pair = [pr.author, comment.author]
22     author_pair.sort()
23     links[author_pair] += 1

```

Listing 1: Algorithm to extract interactions

Listing 1 shows the algorithm to extract interactions from pull requests (PR). When PRs are given, the algorithm iterates all PRs to extract information for the visualisation. If a timeframe is given to the algorithm, it only uses the PRs created within the timeframe. Otherwise, the algorithm leverages the given PRs.

For the PRs included in the scope of the visualisation (i.e., no timeframe is given or PRs created within the timeframe), the algorithm iterates all comments in them. If the PR author and comment author are same, the algorithm skips the comment to avoid counting self-comments. On the other hand, it increases the comment count (i.e., nodes) by one. This approach helps us to extract the contributions by each developer based on the number of comments on PRs that were not authored by themselves. The number of comments will be used to represent the size of each developer node in our visualisation. In addition, we construct a list that contains both PR and comment authors, then increase the count by one. To avoid unnecessary duplicates (e.g., {JohnDoe, JaneDoe} and {JaneDoe, JohnDoe}), we sort the list before increasing the count. The count of author pairs (i.e., links) will be used to control the thickness between developer nodes in the visualisation.

3 Visualisation Framework

The visualisation framework consists of three components as shown in Figure 1: Crawler, Migrator, and Visualiser. The crawler retrieves PRs and comments from GitHub. Then, migrator parse the data and store it to a database. Finally, a user can see a visualisation of a project via the web based visualiser. In this section, we explain the role of each component and implementation details. The implementation details can be found in our replication package³.

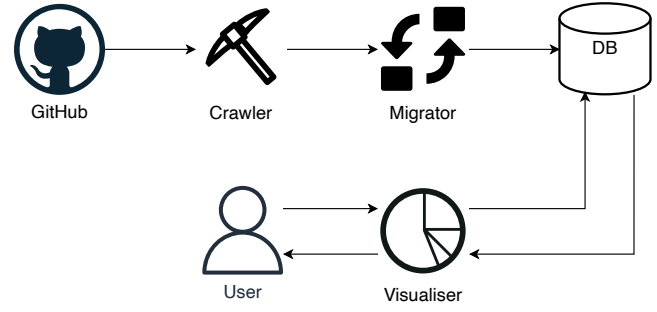


Figure 1: Overall framework

3.1 Crawler

Table 1: Data Properties

Property	Description
number	Numeric id for pull request
author.login	Unique identifiers of developers
author.avatarUrl	URL of developer profile picture
author.__typename	User type (either human user or bot)
createdAt	Timestamp of PR/comment

To visualise developer interactions, we need a large amount of pull request data. It will introduce huge traffics to code review platform if we request data whenever we need them. Such huge traffics potentially slow down the code review platform, so we do not query directly to the code review platform. Instead, we implemented a crawler that can crawl the latest code review data. The crawler leverages GitHub GraphQL API⁴ and implemented in Python. We constructed a query that retrieves author and createdAt properties of PRs and PR comments. Table 1 lists the properties that we crawled.

3.2 Migrator

GraphQL is powerful approach handling large scale requests, but using GitHub GraphQL API directly for visualisation has three outstanding issues. First, GraphQL has additional types of nodes to optimise queries. For example, both PR and comment nodes should be surrounded by the edges and node(s) nodes which introduce unnecessary depth navigation. Second, GraphQL supports only limited conditional query. One of our visualiser's features supports visualising developer interactions within a specific time window. However, GraphQL does not support query that returns PRs that were authored within a specific time range. Last, traversing all PRs take time and require high computation power on the PR platform.

To mitigate the issues, we implemented a migrator that migrates the crawled API responses to a relational database instance. Figure 2 presents an entity relationship diagram of our database schema. The schema represents PRs and comments with the minimum subset of data that is required for visualisation. The migrator was also written in Python, and we evaluated it with a MariaDB instance⁵.

³<https://github.com/DanJBee/CodeReviewVisualisation>

⁴<https://docs.github.com/en/graphql>

⁵<https://mariadb.com>

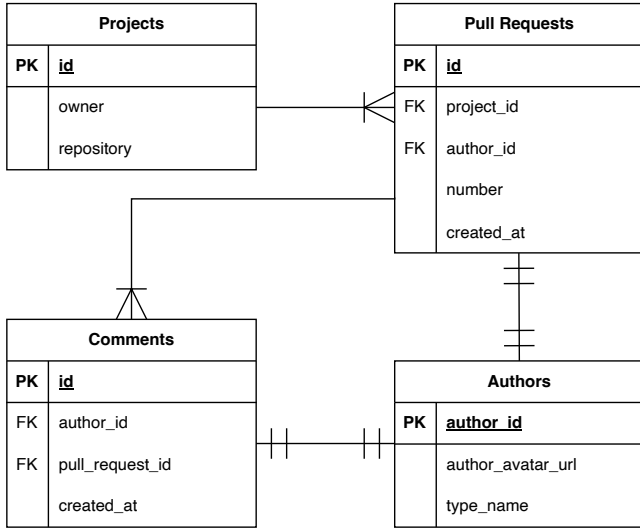


Figure 2: Entity Relationship Diagram

3.3 Visualiser

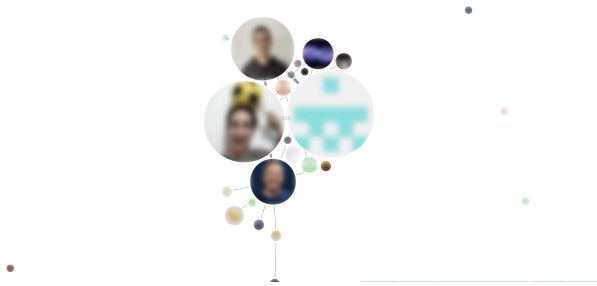


Figure 3: An example of visualisation. We blurred the actual profile photos to protect developers' privacy

The visualiser is responsible for visualising developer interactions. Users can select a specific project and time window to visualise developer interactions. For example, a user can request the visualiser to visualise developer interactions in the TypeScript project for a week from 3rd June 2024.

We implemented the visualiser as a web service that consists of two main components: backend and frontend. The backend component was implemented by using Java and Spring Framework. It retrieves data from the database upon request and run the algorithm described in Section 2. The results are serialised into a JSON string and return to the frontend. The frontend component is implemented by using TypeScript and React Framework. Since developer interactions can be visualised as a graph, we adopted the Force-directed graph component⁶ in D3.

Figure 3 shows an example of the visualisation. Based on the algorithm presented in Section 2, the size of each node represents the number of comments written by each developer with their

⁶<https://observablehq.com/@d3/force-directed-graph>

avatar image on GitHub. Please note that we blurred images on the paper, but real implementation shows clear images. The thickness of edges between nodes denote the number of comments shared by the two developers within the same PRs. To handle extremely large or small cases, we use normalised values for both nodes and edges.

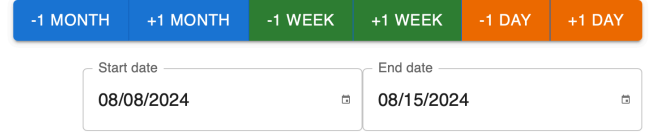


Figure 4: Date Controller

On the right bottom of the frontend, we put a date controller as shown in Figure 4. The user can select specific date by clicking the date field. Once they select a pair of dates, they can easily update the date scope by clicking the buttons above. Whenever the dates updated, the graph visualisation is also dynamically updated.

4 Related Work

There have been many studies aimed at helping developers find code reviewers. Thongtanunam et al. [10] argued that the developers working on large systems struggle to find appropriate reviewers. They assume that a developer who has previously changed the same file could be a good candidate as a reviewer, as they already understand the details of the file. Based on the assumption, they proposed a reviewer recommendation technique called RevFinder. Zanzani et al. [13] also proposed an automated reviewer recommendation approach that leverages previously completed reviews. These approaches are useful for developers to find potential reviewers. However, they do not provide insight of the current review practices. We believe our approach can provide additional information regarding the reviewer recommendation results.

In addition, many empirical studies have been conducted to understand developer interactions during software development. Hong et al. [4] investigated developer social network (DSN) of Mozilla projects and its evolution. They also compared the structure of DSN with general social networks (GSNs) such as Facebook and Twitter. Thung et al. [11] empirically studied social network structure on GitHub. They constructed developer-developer and project-project relationship graphs and identified influential developers and projects by using PageRank. However, their analysis was conducted at a project level, not at the code review level, and they did not provide a tool to visualise the graphs. Bock et al. [2] proposed a method that automatically identifies core developers by using role permissions of privileged events triggered in GitHub issues and pull requests.

5 Potential Use Cases

We believe that our system has several use cases and real-world applications and will make it easier to make data-driven decisions for a project. For instance, it is paramount in identifying inequitable workloads in a development team. Our visualisation helps developers identify members of the team who are reviewing or being reviewed more than others [6].

Specifically, it can be a great help to open source projects. In Figure 3, we can observe several independent nodes that are separated from the graph. These independent nodes in the visualisation indicate that they have contributed to the project, but no one has interacted with them during observed the time period. In our preliminary experiments with multiple projects, we repeatedly observed the independent developers who could not connect to the main graph network regardless of the length of timeframe. This likely indicates a potential scenario in which several newcomers joined the project, but no one welcomed them, leading them left. If the project developers recognise the issue via our visualisation, they could encourage the newcomers to stay involved in the project.

By using the visualisation, we expect developers can plan their code review practices in a better way. If the nodes show a star topology (i.e., a developer participated in all discussions, and there are no interactions between other peer developers) as shown in Figure 3, the team can consider distributing the workload of the developer who located at the centre of the discussion and fostering discussions between peers. In this way, the development team can plan to manage potential risks if the core developer leaves the project.

A development team can consider refreshing their code review practices based on the visualisation. If a pair of developers has been working together for a long time, the team could consider pairing them with a new developer who has worked on the same project, but rarely collaborates with them. These new pairings may help foster creativity among developers.

In addition, the visualisation could help enhance developer recognition and motivation and reduce resentment between members of a team. The size of each developer's contributions (i.e., the size of the nodes in the graph) are a useful quantitative measure of the work balance within the team [5]. Developers are encouraged to knowledge share more frequently as a result of information silos discovered through analysis of graph nodes. Node edge thicknesses help supplement how much intelligence is being shared between team members. Moreover, the code review process can be optimised more thoroughly and the necessary improvements in team dynamics can be made[3, 9].

6 Conclusion

In this paper, we proposed CRV, a web service that visualises developer interactions during code reviews. CRV visualises developer activities and relationships during code reviews in a graph representation. Each node in the graph represents an individual developer, with the node size indicating the frequency of their contributions to code review discussions. Conversely, each edge represents a collaborative relationship between a pair of developers with the edge thickness denoting the frequency of their collaboration within the same PRs.

To provide visualisation without any potential side effects (e.g., huge traffic to the code review platform), we have implemented three components: crawler, migrator and visualiser. Crawler leverages GitHub GraphQL API to retrieve PR data, including code review discussions, from GitHub. Then, migrator parses the retrieved data and store it in a database for flexible and faster querying. Finally, visualiser fetches the data from the database and generates a visualisation based on the specified time frame.

As future work, we plan to conduct an empirical study to investigate the impact of the code review visualisation in practice. We anticipate our approach will lead to a better understanding of code review practice. With this improved understanding, developers may be able to distribute heavy workloads more effectively among their peers or foster the onboarding of hidden newcomers to their projects.

References

- [1] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*.
- [2] Thomas Bock, Nils Alznauer, Mitchell Joblin, and Sven Apel. 2023. Automatic Core-Developer Identification on GitHub: A Validation Study. *ACM Transactions on Software Engineering Methodology* (2023).
- [3] Roy Gelbard and Abraham Carmeli. 2009. The interactive effect of team dynamics and organizational support on ICT project success. *International Journal of Project Management* 27, 5 (2009), 464–470.
- [4] Qiaona Hong, Sunghun Kim, S.C. Cheung, and Christian Bird. 2011. Understanding a developer social network and its evolution. In *2011 27th IEEE International Conference on Software Maintenance (ICSM)*.
- [5] John W Koning Jr. 1993. Three other R's: Recognition, reward and resentment. *Research-Technology Management* 36, 4 (1993), 19–29.
- [6] Mohsin Malik, Shagufta Sarwar, and Stuart Orr. 2021. Agile practices and performance: Examining the role of psychological empowerment. *International Journal of Project Management* 39, 1 (2021), 10–20.
- [7] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2014. The Impact of Code Review Coverage and Code Review Participation on Software Quality: A Case Study of the Qt, VTK, and ITK Projects. In *Proc. of the Working Conference on Mining Software Repositories (MSR)*.
- [8] Rodrigo Morales, Shane McIntosh, and Foutse Khomh. 2015. Do Code Review Practices Impact Design Quality? A Case Study of the Qt, VTK, and ITK Projects. In *Proc. of the International Conference on Software Analysis, Evolution, and Reengineering (SANER)*.
- [9] Ronald H Rasch and Henry L Tosi. 1992. Factors affecting software developers' performance: An integrated approach. *MIS quarterly* (1992), 395–413.
- [10] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-ichi Matsumoto. 2015. Who should review my code? A file location-based code-reviewer recommendation approach for Modern Code Review. In *IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*.
- [11] Ferdian Thung, Tegawendé F. Bissyandé, David Lo, and Lingxiao Jiang. 2013. Network Structure of Social Coding in GitHub. In *2013 17th European Conference on Software Maintenance and Reengineering*.
- [12] Yue Yu, Huaimin Wang, Gang Yin, and Tao Wang. 2016. Reviewer recommendation for pull-requests in GitHub: What can we learn from code review and bug assignment? *Information and Software Technology* (2016).
- [13] Motahareh Bahrami Zanjani, Huzefa Kagdi, and Christian Bird. 2016. Automatically Recommending Peer Reviewers in Modern Code Review. *IEEE Transactions on Software Engineering* (2016).